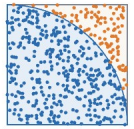
# Calcular π con dardos al azar *y más rápido con hilos*

Les cuento cómo calculé π con el método de Monte Carlo y cómo lo aceleré repartiendo el trabajo entre hilos con OpenMP.

## 1 Lancé dardos

### El método: calcular π con dardos al azar

> *«Conté los dardos que cayeron dentro del círculo, dividí entre el total y multipliqué por 4.»*



**π ≈ 4 × dentro ÷ total**

4 × 78.541.707 ÷ 100.000.000
**= 3,141668**

**Método de Monte Carlo:** usar el azar para calcular

**Palabras clave**

Monte Carlo · π/4 · C · error ∝ 1/√N

## 2 Repartí el trabajo

### Lo que mejoré: repartir el trabajo entre hilos

**Antes:** 1 solo hilo → **1,25 s**

> *«Como **ningún dardo depende de otro**, los repartí.»*

```
#pragma omp parallel for
→ reparte el trabajo
reduction(+:dentro)
→ suma lo de cada hilo
```

> *«Hice **1.000 paquetes** con números distintos.»*

Semilla: **2224510**

**Palabras clave**

OpenMP · parallel for · reduction

## 3 Comprobé

### La prueba: ¿dio siempre el mismo resultado?

| Hilos | Aciertos | Tiempo |
|---|---|---|
| 1 | 78.541.707 | 1,25 s |
| 2 | 78.541.707 | 0,68 s |
| 4 | 78.541.707 | 0,37 s |
| 8 | 78.541.707 | 0,25 s |

*«Siempre obtuve **78.541.707**; solo bajó el tiempo.»*

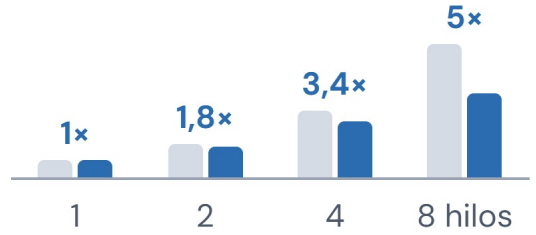
*«Sin reduction, los hilos se pisaron al sumar y π dio **0,65**.»*

**Palabras clave**

reproducible · condición de carrera

## 4 Medí la velocidad

### La ganancia: ¿cuánto más rápido fue?




> *«El programa pasó de **1,25 s** a **0,25 s**: 5 veces más rápido. No llegó a 8 porque mi computador tiene **4 núcleos reales**.»*

**Ley de Amdahl:** una parte del trabajo no se puede repartir; por eso, más hilos ayudan cada vez menos.

**Palabras clave**

speedup · Ley de Amdahl

**Conclusión:** **más hilos → más rápido** | **más dardos → más exacto**

Nancy Liliana Saenz Moreno · Código 2224510 · Computación Paralela, UIS · Medido en mi portátil Ryzen 5 3450U con gcc + OpenMP · 4 núcleos y 8 hilos